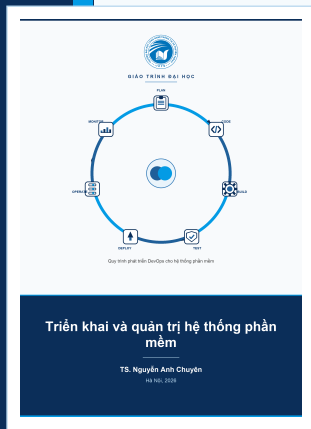




CHƯƠNG 7

Giám sát hệ thống máy chủ

Triển khai và quản trị hệ thống phần mềm



Giới thiệu nội dung

Mục tiêu chương

- Giải thích vai trò giám sát và các nhóm chỉ số hạ tầng, ứng dụng.
- Phân biệt mô hình Pull và Push; trình bày kiến trúc Prometheus và vai trò PromQL.
- Triển khai Docker Compose hệ giám sát Prometheus/Grafana cho WordPress/MySQL với hai network tách biệt.

Nội dung chính

- 1 Tổng quan về giám sát hệ thống**
- 2 Prometheus và Grafana**
- 3 Thực hành: giám sát WordPress + MySQL**

“Chạy được” có chắc “chạy tốt, chạy an toàn”?

- Hệ thống thông tin hiện nay là **huyết mạch sống còn** của doanh nghiệp: web, dịch vụ trực tuyến, hạ tầng mạng, cơ sở dữ liệu.
- Nếu không theo dõi chặt chẽ, doanh nghiệp đối mặt với “**sự cố ẩn**”: tốc độ chậm dần, tài nguyên bị chiếm dụng âm thầm, độ trễ tăng – cho đến khi hệ thống sụp đổ.
- Giám sát (monitoring) không phải tùy chọn thêm thắt, mà là **phần không thể thiếu** của hạ tầng hiện đại.

Câu hỏi mở đầu

Làm sao biết trang web đang tải chậm, người dùng đang gặp lỗi thanh toán, hay bao nhiêu khách truy cập cùng lúc – nếu không có dữ liệu theo dõi?

Hậu quả khi thiếu giám sát

- **Mù mờ về hiệu suất:** không phát hiện được tình trạng chậm dần, tài nguyên quá tải cho đến khi sự cố xảy ra.
- **Lập kế hoạch yếu:** thiếu dữ liệu xu hướng dẫn đến capacity planning theo cảm tính – lãng phí tài nguyên hoặc quá tải.
- **Điều tra sự cố khó khăn:** không có dữ liệu lịch sử để truy nguyên nhân gốc rễ (root cause analysis), kéo dài thời gian gián đoạn dịch vụ.
- **Rủi ro an ninh:** bỏ lỡ các dấu hiệu truy cập trái phép hoặc hành vi bất thường lẽ ra có thể phát hiện sớm qua nhật ký và ngưỡng cảnh báo.

Ví dụ trực quan: giám sát như quản lý kho nhà hàng

Mô hình quản lý nhà hàng

Không giám sát: bếp cứ nấu đến khi khách gọi “bò bít tết” mới phát hiện hết nguyên liệu – khách phải đợi lâu hoặc bỏ đi.

Có giám sát: cảm biến kho báo khi thịt bò xuống dưới 5kg – quản lý chủ động đặt hàng trước khi hết. Nhà hàng vận hành trơn tru.

- Hệ thống giám sát máy chủ chính là “bộ cảm biến kho” đó: biết trang web nhanh hay chậm, lỗi thanh toán hay không, tải hiện tại là bao nhiêu.

Ba lợi ích cốt lõi của giám sát

- Chuyển đổi từ trạng thái **phản ứng thụ động (reactive)** sang **chủ động phòng ngừa (proactive)**.
- Là nền tảng dữ liệu cho **tối ưu chi phí** và **lập kế hoạch hạ tầng dài hạn**.
- Khả năng quan sát được (**observability**) là yếu tố sống còn cho các mô hình cloud-native và microservices phức tạp.

Chỉ số hạ tầng cần thu thập (Infrastructure Metrics)

- **CPU Usage:** theo dõi theo nhãn `user`, `system`, `iowait`; `iowait` cao thường báo hiệu nghẽn cổ chai ổ đĩa.
- **Memory (RAM) Usage:** chú ý cache và swap – dùng nhiều swap là dấu hiệu RAM đã cạn kiệt.
- **Disk Capacity & I/O:** dung lượng trống và tốc độ đọc/ghi (IOPS) để tránh ngưng trệ khi đầy đĩa.
- **Network Throughput:** băng thông, tỷ lệ lỗi gói tin, độ trễ mạng.

Chỉ số ứng dụng cần thu thập (Application Metrics)

- **Request Count:** tổng số yêu cầu HTTP mỗi giây – phản ánh tải trọng thực tế.
- **Error Rate:** tỷ lệ yêu cầu thất bại (HTTP 500, 404...); lỗi 5xx tăng đột biến là tín hiệu khẩn cấp.
- **Latency (Response Time):** thời gian xử lý một yêu cầu – chỉ số ảnh hưởng trực tiếp đến trải nghiệm người dùng.

Điểm cốt lõi

Một ứng dụng có thể “sống” (up) nhưng nếu phản hồi mất 10 giây thì coi như đã “chết” đối với người dùng.

Mô hình dữ liệu đa chiều: metric + labels

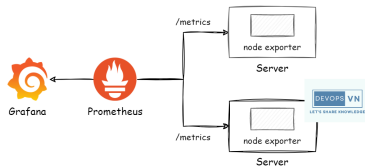
- Prometheus định danh mỗi chỉ số bằng **tên metric** và các cặp **nhãn (labels)** key-value.

```
<metric name>{<label name>=<label value>, ...} <samples>  
http_requests_total{method="POST", endpoint="/login", status="500"}
```

- Nhờ nhãn `method`, `endpoint`, `status`, có thể truy vấn chính xác:
“Có bao nhiêu yêu cầu POST vào `/login` bị lỗi 500?”

Node Exporter: thu thập chỉ số hạ tầng trên Linux

- Công cụ phổ biến nhất để lấy chỉ số phần cứng trên Linux.
- Tự động expose endpoint `/metrics` tại cổng **9100**.
- Prometheus định kỳ **scrape** endpoint này để lấy hàng trăm chỉ số phần cứng.



Mô hình Pull (kéo): cơ chế mặc định của Prometheus

- Prometheus Server **chủ động** gửi yêu cầu HTTP đến các target theo chu kỳ `scrape_interval` để lấy metrics.
- **Quyền kiểm soát:** server quyết định khi nào và tần suất lấy dữ liệu – tránh bị “ngập lụt” nếu nhiều ứng dụng đẩy dữ liệu cùng lúc.
- **Health check dễ dàng:** không kéo được dữ liệu → target “down” ngay lập tức (`up = 0`).
- **Cấu hình target đơn giản:** target chỉ cần mở endpoint HTTP chứa metrics thô, không cần biết Prometheus ở đâu.
- **Service Discovery:** hoạt động hoàn hảo trong môi trường động như Kubernetes – tự tìm container mới mà không cần cấu hình thủ công.

Mô hình Push (đẩy): khi nào cần Pushgateway?

- Một số kịch bản Prometheus **không thể** chủ động kéo dữ liệu, ví dụ các **short-lived jobs** (script sao lưu, dọn dẹp) hoàn thành và tắt trước lượt scrape kế tiếp.
- **Cơ chế:** job chủ động đẩy dữ liệu về **Pushgateway**; sau đó Prometheus kéo dữ liệu từ Pushgateway về như bình thường.
- **Ứng dụng thực tế khác:** máy chủ nằm sau firewall ngăn kết nối đi vào, nhưng vẫn được phép đẩy dữ liệu đi ra.

So sánh Pull và Push

Tiêu chí	Pull (mặc định)	Push (qua Pushgateway)
Ai chủ động	Prometheus Server	Ứng dụng / job
Phù hợp cho	Service chạy liên tục, hạ tầng động (Kubernetes)	Short-lived jobs, mạng bị giới hạn bởi firewall
Health check	Tự động qua <code>up = 0</code>	Không trực tiếp

Kết hợp thực tế

Kiến trúc giám sát toàn diện thường dùng Pull cho service chạy liên tục, và Push chỉ cho các tiến trình nghiệp vụ ngắt quãng.

Case study: Pull trong môi trường Kubernetes động

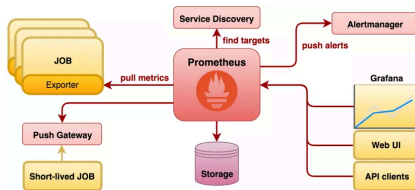
Pod tự khởi động lại liên tục

Khi một pod lỗi và khởi động lại, IP của nó thay đổi. Nhờ Service Discovery, Prometheus (mô hình Pull) tự động cập nhật target mới và tiếp tục kéo dữ liệu **không cần can thiệp thủ công**.

- Nếu dùng Push truyền thống, mỗi lần đổi IP có thể gây rắc rối lớn trong định tuyến dữ liệu về server trung tâm.
- Đây là lý do Pull được xem là lựa chọn hiện đại, phù hợp hơn cho hạ tầng microservices.

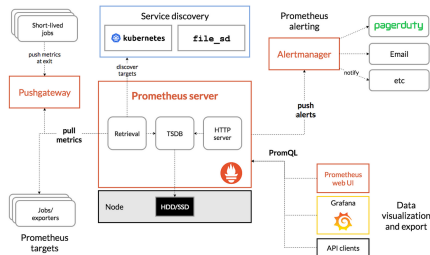
Prometheus: tiêu chuẩn vàng cho giám sát hệ thống phân tán

- Hệ thống giám sát và cảnh báo mã nguồn mở, khởi xướng bởi SoundCloud, là dự án thứ hai gia nhập CNCF sau Kubernetes.
- Tập trung hoàn toàn vào dữ liệu **time-series metrics**, khác với các công cụ dựa trên logs.
- Mỗi Prometheus Server là một thực thể **tự chủ**, lưu dữ liệu cục bộ trong TSDB riêng.



Kiến trúc Prometheus

- **Prometheus Server:** thu thập (Retrieval), lưu trữ (TSDB) và phục vụ truy vấn qua HTTP server.
- **Exporter:** công cụ nhỏ cạnh ứng dụng, chuyển dữ liệu sang định dạng Prometheus hiểu được.
- **Pushgateway:** nhận dữ liệu từ short-lived jobs.
- **Alertmanager:** nhận cảnh báo, nhóm/lọc, điều hướng tới Email, Slack, PagerDuty...



PromQL và giới hạn lưu trữ dữ liệu

- **PromQL** (Prometheus Query Language): tính tỷ lệ yêu cầu mỗi giây (`rate`), phần trăm sử dụng tài nguyên, dự báo xu hướng đầy ổ đĩa...
- **Alerting rules**: tự động nhận diện điều kiện bất thường (vd. CPU > 90% trong 5 phút) và kích hoạt xử lý sự cố.
- Phiên bản 2.0 có thể thu thập hàng triệu mẫu/giây, nén chỉ 1.3 byte/mẫu.
- Thời gian lưu trữ mặc định **15 ngày**; có thể mở rộng qua remote read/write tới Thanos, InfluxDB.

Lưu ý cấu hình

Lưu quá nhiều dữ liệu trên ổ đĩa tốc độ thấp (`retention.time` lớn) làm giảm hiệu suất truy vấn PromQL. Dùng `promtool` kiểm tra cú pháp `prometheus.yml` trước khi khởi động lại dịch vụ.

Case study: chuyển đổi giám sát tại một ngân hàng số

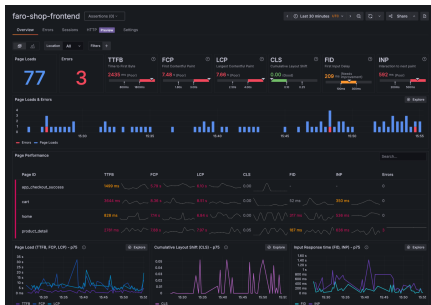
Từ SNMP/logs sang Prometheus + Kubernetes

Khi hệ thống mở rộng tới hàng ngàn microservices, công cụ cũ (SNMP, logs) bị nghẽn cổ chai. Sau khi triển khai Prometheus với Service Discovery, ngân hàng đã:

- Tự động phát hiện và giám sát 100% các pod mới sinh ra.
- Giảm 70% cảnh báo nhiễu nhờ cơ chế nhóm của Alertmanager.
- Phát hiện sớm rò rỉ bộ nhớ (memory leak) qua biểu đồ JVM metrics.

Grafana: “đôi mắt” của hệ thống giám sát

- Nền tảng mã nguồn mở hàng đầu về trực quan hóa dữ liệu; biến số liệu thô thành **dashboard** sinh động.
- Kết nối đồng thời nhiều nguồn dữ liệu: Prometheus, MySQL, PostgreSQL, Elasticsearch, InfluxDB, CloudWatch, Loki...
- Cho phép tạo “single pane of glass” – một màn hình quan sát toàn cảnh hệ thống.



Panels, Variables và Alerting trực quan trong Grafana

- **Panels:** đơn vị hiển thị – biểu đồ đường, cột, gauge, bảng số liệu, heatmap; có thể tô màu đỏ khi vượt ngưỡng an toàn.
- **Variables:** một dashboard duy nhất, lọc dữ liệu theo máy chủ/vùng/dịch vụ qua danh sách thả xuống – không cần tạo hàng chục dashboard riêng.
- **Alerting Engine riêng:** vẽ ngưỡng ngay trên biểu đồ, gửi cảnh báo qua Slack/Discord mà không cần viết PromQL phức tạp.
- **RBAC:** phân quyền Admin/Editor (chỉnh sửa) và Viewer (chỉ xem) cho từng thành viên.

Case study: “war room” giám sát ngày Black Friday

Sàn thương mại điện tử giám sát cao điểm

Một sàn thương mại điện tử dựng “war room” với màn hình lớn hiển thị: đơn hàng thành công theo thời gian thực, tỷ lệ lỗi thanh toán, tải trọng CDN/web.

- Khi một cổng thanh toán treo, panel chuyển đổi ngay lập tức – đội kỹ thuật kịp chuyển hướng luồng thanh toán sang cổng dự phòng, cứu hàng tỷ đồng doanh thu.

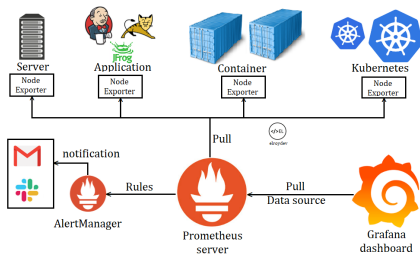
Lưu ý khi triển khai Grafana

Sai lầm thường gặp và bảo mật

- Tạo quá nhiều biểu đồ trên một dashboard gây “ô nhiễm thông tin” (information overload), chậm tải trang, rối mắt – nên chia dashboard chuyên biệt theo từng tầng (hạ tầng, database, application).
- Luôn đổi mật khẩu mặc định `admin/admin` ngay sau khi cài đặt để tránh bị xâm nhập trái phép.

Prometheus + Grafana: hai nửa bổ sung nhau

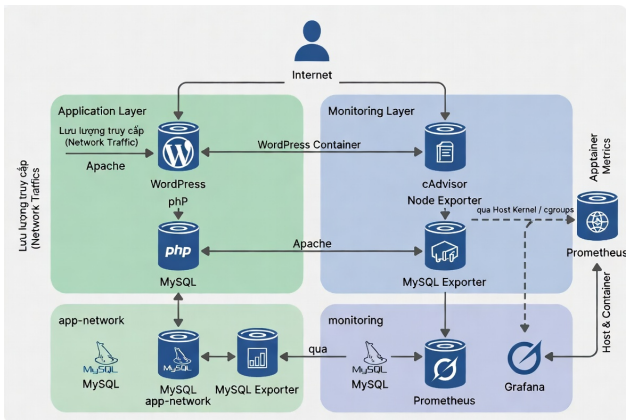
- **Prometheus** = “bộ não”: thu thập, lưu trữ time-series metrics qua Pull và PromQL.
- **Grafana** = “giao diện”: hiển thị dữ liệu trực quan, hỗ trợ cộng đồng dashboard phong phú.
- Kết hợp cả hai giúp doanh nghiệp chuyển từ giám sát thụ động sang quản trị chủ động, phát hiện rủi ro trước khi thành sự cố.



Kịch bản thực hành: giám sát WordPress + MySQL

- Ứng dụng: **WordPress** (Apache/Nginx + PHP) kết hợp CSDL **MySQL** riêng.
- Monitoring stack: **Prometheus** (thu thập), **Grafana** (dashboard), **cAdvisor** (tài nguyên container), **Node Exporter** (host), **MySQL Exporter** (queries, connections, replication).
- Toàn bộ triển khai trên **Docker Compose**, dùng **2 network độc lập**: một cho ứng dụng, một cho giám sát – đảm bảo an toàn dữ liệu.

Sơ đồ kết nối các thành phần của hệ thống



Xanh lá: layer ứng dụng (WordPress + MySQL) **Xanh dương:** layer giám sát

Nét liền: kết nối mạng bình thường Nét đứt: thu thập metrics qua kernel/host (cAdvisor)

mysql-exporter: nằm ở cả hai network

docker-compose.yml – tầng ứng dụng

```
services:
  wordpress:
    image: wordpress:php8.2-apache
    ports: ["8080:80"]
    environment:
      WORDPRESS_DB_HOST: mysql:3306
      WORDPRESS_DB_USER: wpuser
    networks: [app-network]
    depends_on: [mysql]

  mysql:
    image: mysql:8.0
    environment:
      MYSQL_DATABASE: wpdb
      MYSQL_USER: wpuser
    networks: [app-network]
```

- `wordpress` và `mysql` chỉ tham gia mạng `app-network`.

docker-compose.yml – tầng giám sát

```
prometheus:
  image: prom/prometheus:latest
  ports: ["9090:9090"]
  volumes:
    - ./prometheus.yml:/etc/prometheus/prometheus.yml
  networks: [monitoring]

grafana:
  image: grafana/grafana:latest
  ports: ["3000:3000"]
  networks: [monitoring]

mysql-exporter:
  image: prom/mysql-exporter:latest
  environment:
    DATA_SOURCE_NAME: "wpuser:wppass@tcp(mysql:3306)/wpdb"
  networks: [app-network, monitoring] # <- hai network
```

prometheus.yml – khai báo target scrape

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'node-exporter'
    static_configs:
      - targets: ['node-exporter:9100']

  - job_name: 'cadvisor'
    static_configs:
      - targets: ['cadvisor:8080']

  - job_name: 'mysql'
    static_configs:
      - targets: ['mysql-exporter:9104']
```

- Target khai báo bằng **tên service** Docker, không phải `localhost`, vì các container giao tiếp qua DNS nội bộ của Docker network.

Hai network độc lập: vai trò và lý do tách biệt

Mục đích	app-network	monitoring
Chứa service	WordPress + MySQL	Prometheus, Grafana, cAdvisor, Node/MySQL Exporter
Mục tiêu	Traffic ứng dụng (web ↔ DB)	Traffic monitoring
Lý do tách	Giới hạn bề mặt tấn công	Không cần truy cập DB trực tiếp

Lợi ích chính

MySQL không bị expose ra mạng monitoring; Prometheus/Grafana không thể kết nối trực tiếp vào MySQL – tránh rủi ro bảo mật, dễ quản lý và mở rộng.

mysql-exporter: cầu nối giữa hai network

- `mysql-exporter` nằm trên **cả hai** network – đây là cơ chế **multi-network attachment**: một container có thể tham gia nhiều Docker network cùng lúc.
- Dùng `DATA_SOURCE_NAME` để kết nối tới `mysql:3306` qua `app-network`.
- Prometheus (trên mạng `monitoring`) scrape metrics từ `mysql-exporter:9104`.

Điểm cốt lõi

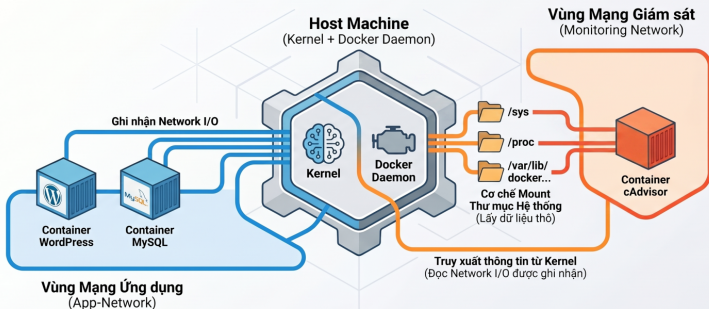
Chỉ những service thực sự cần đọc dữ liệu ứng dụng mới tham gia cả hai network – giữ nguyên tắc giới hạn bề mặt tấn công.

Vì sao cAdvisor không cần app-network vẫn thu thập được traffic?

- Docker Network chỉ dùng để container giao tiếp với nhau (vd. WordPress gọi MySQL) – **không** phải kênh cAdvisor dùng để lấy metrics.
- cAdvisor thu thập qua **cgroup** và **Docker Daemon** ở mức host: đọc trực tiếp `/sys/fs/cgroup` và socket `/var/run/docker.sock`.
- Do chạy ở mức kernel/host, cAdvisor thấy được **mọi container** trên cùng Docker host, bất kể chúng thuộc network nào.

Cơ chế thu thập traffic của cAdvisor

Kiến trúc Giám sát Docker: Phân tách Mạng và Cơ chế Hoạt động của cAdvisor



© NotebookLM

Prometheus thu thập metrics từng service như thế nào?

Service	Cách Prometheus thu thập metrics	Network sử dụng
MySQL	Qua <code>mysql-exporter</code>	<code>monitoring</code> + <code>app-network</code>
WordPress	Gián tiếp qua cAdvisor (network I/O, CPU, RAM)	<code>monitoring</code>
Host / Containers	Qua cAdvisor và Node Exporter	<code>monitoring</code>
Prometheus itself	Tự scrape	<code>monitoring</code>

Tích hợp Grafana với Prometheus làm Data Source

- 1 Configuration (bánh răng) → Data Sources → Add data source → chọn Prometheus.
- 2 URL: `http://prometheus:9090` – dùng **tên service** vì cùng Docker network, không dùng `localhost`.
- 3 Access: Server (mặc định); Scrape interval: 15s (khớp `prometheus.yml`).
- 4 Nhấn Save & Test – thành công khi hiện “Data source is working”.

Import dashboard có sẵn thay vì tự thiết kế

- Dashboards → New → Import.
- Dán ID vào ô → Load.
- Chọn Data Source là Prometheus vừa tạo → Import.

Mục tiêu	ID
Node Exporter	1860
cAdvisor / Docker	14282
MySQL	7362
MySQL Overview	11375

Khởi động và kiểm tra hệ thống

```
$docker compose up -d  
$docker compose ps
```

- Prometheus: `http://localhost:9090`
- Grafana: `http://localhost:3000` (mặc định `admin/admin` – đổi ngay)
- WordPress: `http://localhost:8080`

Tổng kết chương

Những điểm cần ghi nhớ

- Giám sát chuyển hệ thống từ phản ứng thụ động sang chủ động phòng ngừa sự cố.
- Cần thu thập cả chỉ số hạ tầng và ứng dụng; Prometheus lưu dưới dạng time-series có nhãn.
- Pull là mặc định, phù hợp hạ tầng động; Push qua Pushgateway chỉ dành cho job ngắn hạn.
- Kiến trúc Prometheus gồm Server, Exporter, Pushgateway, Alertmanager; Grafana trực quan hóa dữ liệu.
- Đã triển khai giám sát WordPress + MySQL với hai network độc lập để giới hạn bề mặt tấn công.